

Partie 10

Exceptions

Définition et exemple simple

Définition

Un programme peut être confronté à une **condition exceptionnelle** durant son exécution.

Exception

Une **exception** est une situation qui **empêche l'exécution normale** du programme.

- un fichier nécessaire à l'exécution du programme n'existe pas,
- une division par zéro,
- un débordement dans un tableau,
- un besoin de se connecter à un serveur et celui-ci est injoignable,
- un dépilement d'une pile vide,

Empêcher l'exécution normale ?

Que signifie **empêcher l'exécution normale** ? Exemple : lecture d'un fichier de données.

Situation normale :

- ouverture du fichier,
- lecture du fichier,
- fermeture du fichier.

Situation exceptionnelle :

- ouverture du fichier → **échec !**
- **ne peut pas lire le fichier !**
- **ne peut pas fermer le fichier !**

Le traitement normal du fichier n'est pas possible en situation d'exception.

Exception vs. erreur

Une exception peut être une erreur...ou pas. Ce peut aussi être un comportement anormal et rare, mais inévitable par le programmeur.

Erreurs :

- division par zéro (`ArithmeticException`),
- débordement d'un tableau (`ArrayOutOfBoundsException`),
- dépassement de pile (`StackOverflowException`),...

Comportement rare :

- fichier inouvrable (`FileNotFoundException`),
- connexion coupée (`ClosedChannelException`),
- opération système interrompue (`InterruptedException`),...

Gestion des exceptions en Java

Comment une méthode doit gérer une exception ?

- elle ne retourne pas,
- elle **crée** un objet exception (étendant `Exception`),
- elle termine en **émettant** cette exception (mot-clé **throw**).
- l'exception est **propagée** à la méthode appelante, qui elle-même termine immédiatement en émettant l'exception, ...
- toute méthode peut **rattraper** l'exception, pour reprendre le calcul normalement après un éventuel correctif.
- Une exception non-rattrapée termine le programme.

Exemple simple

On considère une roulette de casino :

- 36 numéros rouges ou noirs, plus le 0 vert (la banque encaisse tous les paris).
- On tire un numéro aléatoire à chaque partie.
- Les joueurs peuvent parier sur un numéro précis, une couleur, une parité, un groupe de numéros, ...
- On considère le tirage du 0 exceptionnel. Ce n'est pas un bon cas d'usage car ce n'est ni anormal, ni erroné, mais cela va nous permettre d'illustrer les notions sur un cas simple.

Exemple simple

Consulter l'exemple `cours9.MainRoulette`.

- `Roulette.roll` peut émettre une exception `ZeroException`;
- `MainRoulette.playThreeTimes` ignore les exceptions et les propage;
- `MainRoulette.main` rattrape les exceptions `ZeroException`.

Émettre une exception

Comment émettre une exception ?

- mot-clé `throw` avec une instance d'une exception.

```
throw new NoSuchElementException();
```

- `throw` est une instruction terminant l'exécution de la méthode.
- une méthode terminée avec `throw` ne retourne pas de valeur !
- une méthode qui peut terminer avec une exception doit le mentionner dans sa déclaration, avec le mot-clé `throws`.

Méthode subissant une exception

Une méthode `m()` peut exécuter une instruction (par exemple appeler une autre méthode `mayFail()`) qui termine avec une exception. `m()` peut **ignorer** ou **traiter** l'exception.

si `m()` ignore l'exception :

- `m()` termine immédiatement,
- `m()` ne retourne pas de valeur,
- `m()` **propage** l'exception, qui peut être traitée ou ignorée par la méthode ayant fait appel à `m()`,
- `m()` **doit déclarer** avec **throws** qu'elle peut propager l'exception.

Méthode subissant une exception

Une méthode `m()` peut exécuter une instruction (par exemple appeler une autre méthode `mayFail()`) qui termine avec une exception. `m()` peut **ignorer** ou **traiter** l'exception.

si `m()` traite l'exception :

- `m()` **rattrape** l'exception en utilisant la syntaxe **`try catch`**,
- `m()` définit des instructions spécifiques au traitement de l'exception (bloc d'instructions **`catch`**),
- `m()` a accès à l'instance d'exception (qui peut avoir des propriétés et des méthodes portant des informations sur l'exception),
- `m()` **ne déclare pas** propager l'exception.

Rattraper une exception

Pour rattraper une exception :

- les instructions pouvant produire l'exception sont placés dans un bloc précédé du mot-clé `try`,
- après ce bloc, on ajoute un bloc introduit par le mot-clé `catch` et une déclaration d'exception entre parenthèses.
- le bloc `catch` contient les instructions pour traiter le cas exceptionnel.

```
try { // bloc des instructions pouvant produire l'exception
    doSomethingThatMayFail();
}
catch (MyException exc) { // déclaration de l'exception exc
    // bloc des instructions pour traiter l'exception exc
    System.out.println(exc);
}
```

Exemples de sortie

Une partie sans exception :

Pair

Impair

Impair

Une partie interrompue par une exception au deuxième tirage
(le troisième tirage n'a pas lieu) :

Impair

Zéro, la banque prend tout !

La classe Exception

- `throw` ne peut émettre que des instances ayant le type `Throwable`.
- `Exception` et `Error` sont deux extensions de `Throwable`.
- `Error` représente les erreurs sérieuses ne devant pas être rattrapées, `Exception` représente les erreurs et comportements exceptionnels pouvant être rattrapés.
- `Exception` est une classe pouvant être étendue.
- N'importe quelle extension d'`Exception` peut être émise, propagée, rattrapée.
- Les extensions peuvent posséder des propriétés et des méthodes, ce sont des classes normales.

La classe RuntimeException

- RuntimeException est une extension de Exception,
- les exceptions de type RuntimeException ne nécessitent pas de déclaration **throws**,
- c'est le cas de RuntimeException et de ses extensions, comme :
 - ArithmeticException,
 - NoSuchElementException,
 - IllegalArgumentException,
 - NullPointerException,...
- souvent, ces exceptions sont conséquences d'**erreurs du programmeur** !

Usage avancé

finally

- Le mot-clé `finally` permet d'introduire un bloc d'instructions après un `try`.
- Les instructions de `finally` sont `toujours` exécutées, même en cas de retour (`return`) ou d'émission d'exception (`throw`).
- Les instructions sont exécutées quand le contrôle quitte le bloc `try` ou `catch`.
- Les instructions peuvent ne pas être exécutées si le programme (ou le processus) termine (avec `System.exit` par exemple).
- On peut utiliser `try finally` sans utiliser d'exceptions.
- Cas d'usage principal : fermeture des ressources (fichiers, sockets, ...).

Exemple d'usage de finally

Lecture d'entiers dans un fichier :

```
Scanner scanner = new Scanner(Path.of("data.txt"));
List<Integer> ints = new ArrayList<>();
try {
    while (scanner.hasNext()) {
        ints.add(scanner.nextInt());
    }
} catch (InputMismatchException exc) {
    System.out.println("Wrong input at " + ints.size());
} finally {
    scanner.close();
};
```

Exception levée ou pas, le scanner est bien fermé.

Ressource avec Autoclosable

La **fermeture des ressources** est tellement typique qu'il existe une syntaxe adaptée, si la ressource est une classe implémentant `Autoclosable` :

- création de la ressource juste après le **try**,
- la ressource, si elle est ouverte, est automatiquement fermée en fin de bloc.

```
List<Integer> ints = new ArrayList<>();
try (Scanner scanner = new Scanner(Path.of("data.txt"))) {
    while (scanner.hasNext()) {
        ints.add(scanner.nextInt());
    }
} catch InputMismatchException exc) {
    System.out.println("Wrong input at " + ints.size());
}
```

Autre exemple avec Autoclosable

Compter le nombre de lignes d'un fichier :

```
try (BufferedReader reader =  
    Files.newBufferedReader(new File("foo.txt"))  
    )  
{ System.out.println(reader.lines().count()); }  
catch (Exception e) {  
    System.out.println("Something bad happened.");  
}
```

Le fichier est garanti d'être fermé à la fin de la méthode.

Erreur à l'exécution

Une exception non-rattrapée provoque l'arrêt du programme avec un message d'erreur. Exemple :

```
Exception in thread "main" com.univamu.parseFile.ParsingExceptionWithLinesRead
at com.univamu.parseFile.FileParser.readAll(FileParser.java:42)
at com.univamu.parseFile.Main.readFile(Main.java:12)
at com.univamu.parseFile.Main.main(Main.java:29)
Caused by: com.univamu.parseFile.LongLineParsingException
at com.univamu.parseFile.FileParser.readLine(FileParser.java:21)
at com.univamu.parseFile.FileParser.processNextLine(FileParser.java:52)
at com.univamu.parseFile.FileParser.readAll(FileParser.java:37)
... 2 more
```

Analyse du message d'erreur

- Ligne 1 : `ParsingExceptionWithLinesRead` exception à haut-niveau, c'est l'exception non rattrapée,
- Lignes 2 à 4 : la liste des méthodes ayant émise (`readAll`) et propagées (`readFile`, `main`) l'exception
- Ligne 5 : exception bas-niveau `LongLineParsingException`, c'est l'exception qui est à l'origine de l'erreur,
- Lignes 6 à 8 : méthodes l'ayant émise et propagée.
- L'exception bas-niveau est détaillée **parce que** elle a été déclarée comme **cause** de `ParsingExceptionWithLinesRead` dans son constructeur :

```
this.initCause(exc);
```

- Ces informations peuvent aussi être obtenues avec la méthode `printStackTrace` de la classe `Exception`.

Cause d'une exception

Les exceptions peuvent posséder une **cause**,

- de type `Throwable` ;
- accesseur `getCause`, mutateur `initCause` ;
- indique quelle erreur a provoqué en chaîne la production de cette erreur ;
- la première erreur n'a pas de cause (**`null`**).

Initialiser la cause

Lorsque qu'une exception est levée en réponse à une autre exception (dans un `catch`), il faut lui renseigner sa cause :

```
try {  
    ...  
} catch (SomeLowLevelException exc1) {  
    SomeHighLevelException exc2 =  
        new SomeHighLevelException();  
    exc2.initCause(exc1);  
    throw exc2;  
}
```

La trace de la pile d'appels.

- Chaque méthode interrompue à cause d'une exception provoque la suppression d'une couche de la pile en mémoire.
- Les exceptions enregistrent les méthodes interrompues.
- `exc.printStackTrace()` permet d'afficher les méthodes interrompues.
- `exc.getStackTrace()` fournit le tableau des méthodes interrompues (objets de la classe `StackTrace`).

Difficultés supplémentaires.

Que se passe-t-il

- si les instructions d'un `catch` provoque une exception ?
- si les instructions d'un `finally` provoque une exception ?

La sémantique des exceptions est complexe. Conseil :

- limiter l'emploi des exceptions aux cas exceptionnels ;
- garder les instructions des blocs `catch` et `finally` aussi simples que possible.

Récapitulatif

Récapitulatif (I)

- On utilise les exceptions **en cas de comportement empêchant l'exécution normale** du programme.
- Les exceptions sont soumises à un mécanisme en 3 étapes : émission (**throw**), propagation, rattrapage (**catch**).
- Une exception **non-rattrapée** termine le programme.
- Les exceptions sont des objets, instances de `Exception` ou d'une de ces extensions.
- Une méthode qui **émet** ou **propage** des exceptions doit le déclarer (**throws**).
- Une méthode terminée par une exception (émission ou propagation) ne retourne pas de valeur. L'exécution du programme reprend dans un bloc **catch**.

Récapitulatif (II)

- `try catch` permet de rattraper les exception.
- Un `try` peut posséder plusieurs `catch` pour rattraper différents types d'exceptions.
- un `catch` rattrape les exceptions d'un type d'exception et de `tous ses sous-types`. Un `throws` déclare la propagation d'un type d'exception et de tous des sous-types.
- `finally` permet de garantir l'exécution d'instructions lorsque le contrôle sort d'un bloc, d'une méthode.
- Lors de la réémission d'une exception, il faut définir la cause de la nouvelle exception avec la méthode `initCause`.